

Level 1:

1. Write a Python program to accept five statements related to cryptology from the user and store them in a text file named cryptology.txt. Store each statement on a separate line.

sample statements:

Cryptology is the study of secure communication.

Cryptography focuses on protecting information.

Cryptanalysis focuses on analyzing cryptographic systems.

Plaintext represents the original message.

Ciphertext represents the transformed message.

Cryptanalysis is fun :)

I love Cryptanalysis ;)

```
file = open("cryptology.txt", "w")

for i in range(5):
    statement = input("Enter statement: ")
    file.write(statement + "\n")

file.close()

print("Statements stored successfully.")
```

Enter statement: Cryptology is the study of secure communication.
Enter statement: Cryptography focuses on protecting information. Cryptanalysis focuses on analyzing cryptographic systems.
Enter statement: Plaintext represents the original message. Ciphertext represents the transformed message.
Enter statement: Cryptanalysis is fun :)
Enter statement: I love Cryptanalysis ;)
Statements stored successfully.

2. Write a Python program to read cryptology.txt and display its complete contents.

```
file = open("cryptology.txt", "r")

contents = file.read()
print(contents)

file.close()
```

Cryptology is the study of secure communication.
Cryptography focuses on protecting information. Cryptanalysis focuses on analyzing cryptographic systems.
Plaintext represents the original message. Ciphertext represents the transformed message.
Cryptanalysis is fun :)
I love Cryptanalysis ;)

3. Write a program to read cryptology.txt one line at a time and display each line along with its line number.

```
file = open("cryptology.txt", "r")

line_number = 1

for line in file:
    print(line_number, line.strip())
    line_number += 1

file.close()
```

1 Cryptology is the study of secure communication.
2 Cryptography focuses on protecting information. Cryptanalysis focuses on analyzing cryptographic systems.
3 Plaintext represents the original message. Ciphertext represents the transformed message.
4 Cryptanalysis is fun :)
5 I love Cryptanalysis ;)

4. write a program to store the output in a file called cryptology_no.txt

```
file = open("cryptology.txt", "r")
output = open("cryptology_no.txt", "w")

line_number = 1

for line in file:
    output.write(str(line_number) + " " + line)
    line_number += 1

file.close()
output.close()

print("Output stored in cryptology_no.txt")
```

Output stored in cryptology_no.txt

5. Write a program to read cryptology.txt and determine:

Number of lines

Number of words

Number of characters

```
file = open("cryptology.txt", "r")

lines = file.readlines()

line_count = len(lines)
word_count = 0
character_count = 0

for line in lines:
    word_count += len(line.split())
    character_count += len(line)

print("Number of lines:", line_count)
print("Number of words:", word_count)
print("Number of characters:", character_count)

file.close()
```

Number of lines: 5
Number of words: 36
Number of characters: 294

6. Read cryptology_no.txt and count the number of:

Uppercase letters

Lowercase letters

Digits

Spaces

Special characters

```

file = open("cryptology_no.txt", "r")

uppercase = 0
lowercase = 0
digits = 0
spaces = 0
special = 0

for line in file:
    for ch in line:
        if ch.isupper():
            uppercase += 1
        elif ch.islower():
            lowercase += 1
        elif ch.isdigit():
            digits += 1
        elif ch == " ":
            spaces += 1
        elif ch != "\n":
            special += 1

print("Uppercase letters:", uppercase)
print("Lowercase letters:", lowercase)
print("Digits:", digits)
print("Spaces:", spaces)
print("Special characters:", special)

file.close()

```

```

Uppercase letters: 8
Lowercase letters: 240
Digits: 5
Spaces: 37
Special characters: 9

```

7. Write a program that accepts a word from the user and searches for that word in cryptology.txt.

```

word = input("Enter word to search: ")

file = open("cryptology.txt", "r")

found = False

for line in file:
    if word.lower() in line.lower():
        print("Word found:", line.strip())
        found = True

if not found:
    print("Word not found.")

file.close()

```

```

Enter word to search: fun
Word found: Cryptanalysis is fun :)

```

8. Read cryptology.txt and display only the lines containing the word message.

```

file = open("cryptology.txt", "r")

for line in file:
    if "message" in line.lower():
        print(line.strip())

file.close()

```

Plaintext represents the original message. Ciphertext represents the transformed message.

9. Write a Python program to read cryptology.txt, convert all alphabetic characters to uppercase, and store the result in a new file called normalized.txt.

```
file = open("cryptology.txt", "r")
output = open("normalized.txt", "w")

for line in file:
    output.write(line.upper())

file.close()
output.close()

print("Contents stored in normalized.txt")

Contents stored in normalized.txt
```

10. Write a program to read the file and print only the lines containing only alphabetic characters from the cryptology_no.txt file.

```
file = open("cryptology_no.txt", "r")

for line in file:
    parts = line.split(" ", 1)

    if len(parts) == 2:
        text = parts[1].strip()

        if text.isalpha():
            print(text)

file.close()
```

11. Write a program to get the string from the user and reverse the string.

```
string = input("Enter a string: ")
reverse = string[::-1]
print("Reversed string:", reverse)

Enter a string: Cryptanalysis
Reversed string: sisylanatpyrC
```

12. Improve the above code to reverse the string from the file.

```
file = open("cryptology.txt", "r")

text = file.read()

reverse = text[::-1]

print("Reversed string:")
print(reverse)

file.close()

Reversed string:

); sisylanatpyrC evol I
): nuf si sisylanatpyrC
.egassem demrofsnart eht stneserper txetrehpiC .egassem lanigiro eht stneserper txetnialP
.smetsys cihpargotpyrC gnizylana no sesucof sisylanatpyrC .noitamrofni gnitcetorp no sesucof yhpargotpyrC
.noitacinummoc eruces fo yduts eht si ygotlotpyrC
```

13. Simulate the tac command in Python.

```
file = open("cryptology.txt", "r")

lines = file.readlines()

for line in reversed(lines):
    print(line.strip())

file.close()

I love Cryptanalysis ;)
Cryptanalysis is fun :)
Plaintext represents the original message. Ciphertext represents the transformed message.
Cryptography focuses on protecting information. Cryptanalysis focuses on analyzing cryptographic systems.
Cryptology is the study of secure communication.
```

or

Create a Python program that accepts five lines of text related to cryptology from the user and stores them in input.txt. Read the file and determine the number of lines, words, alphabetic characters, digits, spaces, and special characters. Convert the text to uppercase and store the processed contents in processed.txt. Finally, read and display the contents of processed.txt.

Accept five lines and store them in input.txt

```
file = open("input.txt", "w")
```

```
for i in range(5):
```

```
    line = input("Enter line: ")
```

```
    file.write(line + "\n")
```

```
file.close()
```

Read the file and count lines, words, alphabets, digits, spaces and special characters

```
file = open("input.txt", "r")
```

```
lines = file.readlines()
```

```
line_count = len(lines)
```

```
word_count = 0
```

```
alphabetic = 0
```

```
digits = 0
```

```
spaces = 0
```

```
special = 0
```

```
for line in lines:
```

```
    word_count += len(line.split())
```

```
    for ch in line:
```

```
        if ch.isalpha():
```

```
            alphabetic += 1
```

```
        elif ch.isdigit():
```

```

        digits += 1
    elif ch == " ":
        spaces += 1
    elif ch != "\n":
        special += 1
file.close()
print("\nNumber of lines:", line_count)
print("Number of words:", word_count)
print("Alphabetic characters:", alphabetic)
print("Digits:", digits)
print("Spaces:", spaces)
print("Special characters:", special)

```

```

# Convert the text to uppercase and store in processed.txt

```

```

file = open("input.txt", "r")
processed = open("processed.txt", "w")
for line in file:
    processed.write(line.upper())
file.close()
processed.close()

```

```

# Read and display processed.txt

```

```

processed = open("processed.txt", "r")
print("\nContents of processed.txt:")
print(processed.read())
processed.close()

```

```

Enter line: Cryptology is the study of secure communication.
Enter line: Cryptography focuses on protecting information. Cryptanalysis focuses on analyzing cryptographic systems.
Enter line: Plaintext represents the original message. Ciphertext represents the transformed message.
Enter line: Cryptanalysis is fun :)
Enter line: I love Cryptanalysis ;)

```

```

Number of lines: 5
Number of words: 36
Alphabetic characters: 248
Digits: 0
Spaces: 32
Special characters: 9

```

```

Contents of processed.txt:
CRYPTOLOGY IS THE STUDY OF SECURE COMMUNICATION.
CRYPTOGRAPHY FOCUSES ON PROTECTING INFORMATION. CRYPTANALYSIS FOCUSES ON ANALYZING CRYPTOGRAPHIC SYSTEMS.
PLAINTEXT REPRESENTS THE ORIGINAL MESSAGE. CIPHERTEXT REPRESENTS THE TRANSFORMED MESSAGE.
CRYPTANALYSIS IS FUN :)
I LOVE CRYPTANALYSIS ;)

```

Level 2: Regex

security_data.txt:

User alice logged in from 192.168.10.15

User bob logged in from 10.10.5.21

Contact admin@securitylab.com

Failed login attempts: 5

Session ID: SEC-2026-1045

Hash: a94f3c2d8e71b05f

Port 443 connection established

User admin failed authentication

1. Write a Python program to read security_data.txt and use a regular expression to identify and display all IPv4-address-like patterns present in the file.

```
import re

file = open("security_data.txt", "r")
text = file.read()
pattern = r"\b(?:\d{1,3}\.){3}\d{1,3}\b"

ips = re.findall(pattern, text)
for ip in ips:
    print(ip)

file.close()

192.168.10.15
10.10.5.21
```

2. Write a Python program to identify and display all email-address-like patterns present in security_data.txt.

```
import re

file = open("security_data.txt", "r")
text = file.read()
pattern = r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b"

emails = re.findall(pattern, text)

print("Email addresses:")
for email in emails:
    print(email)

file.close()

Email addresses:
admin@securitylab.com
```

3. Write a Python program to extract session IDs that follow the format SEC-YYYY-NNNN, where YYYY and NNNN represent four digits.

```
import re

file = open("security_data.txt", "r")
text = file.read()
pattern = r"\bSEC-\d{4}-\d{4}\b"

session_ids = re.findall(pattern, text)

print("Session IDs:")
for session in session_ids:
    print(session)

file.close()
```

Session IDs:
SEC-2026-1045

4. Write a Python program to identify all hexadecimal sequences containing the characters 0–9, a–f, or A–F.

```
import re

file = open("security_data.txt", "r")
text = file.read()
pattern = r"\b(?:[0-9a-fA-F]*[a-fA-F])[0-9a-fA-F]+\b"

hex_values = re.findall(pattern, text)

print("Hexadecimal sequences:")
for value in hex_values:
    print(value)

file.close()
```

Hexadecimal sequences:
a94f3c2d8e71b05f

5. Write a Python program to read security_data.txt and display all lines containing the word failed, irrespective of uppercase or lowercase.

```
import re

file = open("security_data.txt", "r")

for line in file:
    if re.search(r"\bfailed\b", line, re.IGNORECASE):
        print(line.strip())

file.close()
```

Failed login attempts: 5
User admin failed authentication

6. Write a Python program to identify and extract the port number from statements of the form Port 443 connection established.

```
import re

file = open("security_data.txt", "r")
text = file.read()
pattern = r"Port\s+(\d+)\s+connection established"

match = re.search(pattern, text)
if match:
    print("Port number:", match.group(1))

file.close()

Port number: 443
```

or

Write a Python program to read the contents of security_data.txt and use regular expressions to extract IP-address-like patterns, email addresses, session IDs, and hexadecimal sequences. Store the identified patterns in a new file named pattern_results.txt.

```
import re

file = open("security_data.txt", "r")
output = open("pattern_results.txt", "w")
text = file.read()

# IP addresses
ips = re.findall(r"\b(?:\d{1,3}\.){3}\d{1,3}\b", text)

# Email addresses
emails = re.findall(
    r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b",
    text
)

# Session IDs
sessions = re.findall(r"\bSEC-\d{4}-\d{4}\b", text)

# Hexadecimal sequences
hex_values = re.findall(r"\b[0-9a-fA-F]+\b", text)

output.write("IP Addresses:\n")

for ip in ips:
    output.write(ip + "\n")
```

```
output.write("\nEmail Addresses:\n")
```

```
for email in emails:
```

```
    output.write(email + "\n")
```

```
output.write("\nSession IDs:\n")
```

```
for session in sessions:
```

```
    output.write(session + "\n")
```

```
output.write("\nHexadecimal Sequences:\n")
```

```
for value in hex_values:
```

```
    output.write(value + "\n")
```

```
file.close()
```

```
output.close()
```

```
print("Patterns stored in pattern_results.txt")
```